

Lab02 MoE 的向量化计算

优化 MoE 前向

先阅读了 `moe_ref.cpp` 的实现，首先注意到 S1-S4 场景的参数是固定的，并且它们彼此之间差异较大，所以打算拆分成四个分支分别进行优化。

S1

S1 的特点是 $N=1$ ，并且并行度极小。

肉眼可见地，我们可以先通过向量化方法优化。

对 router 我们在预处理时转置，这样在推理时就能用 `avx512` 一次循环直接计算 16 个 expert 的分数。同时为了避免生成依赖链，我们这里搞多组 accumulator 来算，把计算打满。

对于 gate/up/down 这些部分，我们打算用 `VPDPBUSD`，那这就需把权重打包一下，然后去算。此外因为这里 gate/up 共享同一份后的 token，所以可以融合起来计算，避免重复 load。

然后把 `SiLU` 和 `exp` 等等进行泰勒展开，继续用 `AVX512` 优化。

到这里大概做到了 9x 左右的优化。使用 `perf stat` 分析发现已经有 47% 左右的 membound，考虑能不能优化一下内存访问。

发现每个数据读了之后只会被用一次（换句话说，计算密度是 1MAC/byte），也就是说优化访问模式没啥用。只有 prefetch 可能会有点用。

做完这个之后得到了 10x 左右的优化，此时进行分析，看看我们在单核上有没有优化空间。

权重在 L1 里塞不下，所以内存方面的上限要看 L2。

理论上 $VNNI \text{ compute roof} = 2 * 128 = 256 \text{ ops/cycle}$ ，但是 L2 提供 64 B/cycle，也就是说 $L2 \text{ roof} = 64 \text{ B/cycle} * 2 \text{ ops/B} = 128 \text{ ops/cycle}$ 。所以这是上界，这是一个计算密度低的 case。通常而言要先做 batch，但是我们这是在线问题，没办法，只能硬着头皮继续做。

做了一个 baseline 在集群上测发现 L2 大概只有 48.4 B/cycle，所以实际 roofline 大概也就 48.4 MAC/cycle，按照这个实际数据计算，我们一共有 491520 MAC 操作，至少需要 10155 cycles。

通过工具发现，我们每轮 forward 实际用了 14316 cycles，考虑到 router, `SiLU`, 量化 等的时间消耗，我们可以认为在单核上已经做到头了。

考虑多核，问题出在 `omp` 的 overhead 可能太大（我们一次 forward 只要 5 微秒）。

naive 版本就是每个 expert 开一个 `omp thread`，实测拿到 16x 左右。

显然没有达到我们想要的额外 5x 加速，所以下一步就是打下 `omp thread` 的 overhead。

手写 worker pool，最终做到稳定 23.4x 加速。

S2

跟 S1 类似的配置，不过规模大了，可能可以做一些 `AMX` 或者更大的并行度。

先仿照 S1 做了一轮单核向量化，得到 2.2x 的效果。这是在预期之中的，因为此时单个 expert 的权重来到了 1.5MB，而 L2 cache 只有 2MB，我们有 5 个 expert，在单核上挤不下，肯定会有很大内存开销。

后面想了一下，觉得在单核上基本还是做到头了。考虑如何利用这个更大的并行度，分到 16 cores 上执行。

考虑怎么切分任务。这里当时思考的时候，我忘记了分配的 16cores 是 8 个物理核表现为 16 个逻辑核；我当成分配的是 16 个物理核。这样计算下来，发现 17 个 expert 的权重分到 16 个核刚好是 1.7MB 左右，可以卡进 L2 cache 里。

所以我提出了这样的分法：每个 expert 的 gate/up 各自有 32 个输出 block，down 有 64 个输出 block。我们把它们平均分给 16 个线程，每个线程负责全部 17 个 expert 中的一部分权重，这样可以让计算负载均衡。

实现了一下，并且这里也做了手动控制线程，拿到了 27x 的优化结果。

当然，最后我们肯定是没办法卡到 L2 cache 里的。因为最终也就 8 个 core，权重肯定是放不下的，也就没有进一步进行这方面的尝试。

S3

进入 S3，最重要的变化是 batchsize 变大了。

显而易见地，我们可以使用 AMX 进行优化。我们用一个 tile 存权重，一个 tile 存 token，剩下六个 tile 用来累加，这样算一轮下来可以算完 96 个 token。同时仍然可以使用之前在单核上的优化。这样，我们在单核上就能做到 31x。

考虑怎么做多核。首先还是只有 8 线程是有意义的。其次，这里涉及到一个尽量均分任务的问题。我们把 96 个 token 行作为一轮，对每个 expert 算需要多少轮，并且在轮之间均分 tile。之后对每个任务估算一个 cost，然后按照贪心的方法分配任务给目前累计 cost 最小的线程。

得到了 102x 的优化。

profile 得到的结果，86% backend bound，其中 54.7 membound，31.9 core bound。进一步分析发现主要瓶颈是线程同步。

进一步主要的优化方向是，考虑进一步更细粒度的任务分配方法，以及把 cache 纳入考虑范围内，给 expert 设置偏好线程。

S4

跟 S3 类似，但是计算发现每个 expert 平均分到 2 个 token，密度低，导致直接对 N 进行并行甚至成为一个可行的 baseline。

不过我们还是采用一种更复杂一点的方法。

对于 router 部分，我们仍然使用 AVX + 对 N 进行并行。

对于 expert 部分，我们统计每个 expert 被分到几个 token，设置一个阈值（我选的是 8），低于这个阈值使用 VNNI，高于这个阈值使用 AMX。

划分完任务之后，使用经验算法估计每个任务需要多少周期，然后用 LPT 分配任务。

直接实现这个算法可以得到 80x 左右的优化。

进行 profile，发现约有 40% 的时间花在 router 部分上。

考虑我们到只是我们在 512 个 expert 中选出 2 个来，没必要那么精确，所以我们先量化，然后用 VNNI 来做粗筛，然后再用 fp32 精确计算。这里我们预留 16 个候选。

但是维护 top16 还是很慢。所以修改成在每个 VNNI block 内部直接取 top2，得到 64 个 candidates。

还是不够快。修改成在每个 VNNI block 内部取 top1，然后在 top1 分数最高的 8 个 block 内取 top2，得到 40 个 candidates。

最后得到了 129.8x 的优化效果。

突然发现，我们之前 router 不用 AMX 是因为 AMX 不支持 fp32，现在没有道理不用。

最后结果是 148x。

Bonus: RISC-V

S1/S2/S3/S4

思路没有区别，只是用 RVV, vmadot 实现罢了。

思考题

1

对单个专家来说， $3 \times 256 \times 128 = 98304$ ，也就是说有 96KB 的权重，并且会产生这么多次 MAC。

每个 token 调用 5 个专家，所以 naive 版本中的总访存量是 $128 \times 5 \times 96KB = 60MB$ ，计算量也是这么大。

所以 naive 版本中算术强度是 1MAC/byte，太小了，明显的访存瓶颈。

按专家分组后，总访存量最多只有 $17 \times 96KB = 1.59MB$ ，计算量不变，所以计算强度是 37.65 MAC/byte。变成了计算瓶颈。

分组本身通过减少内存访问来加速。但是对于这个情景来说，所有权重都能放在 L2 cache 内，所以它本身的加速没有那么夸张。

事实上，分组之后变成计算瓶颈，我们可以把分组后的结果打包给 AMX 进一步加速。这是分组带来的间接加速。

2

对于矩阵来说，肯定也是 per-channel 用一个 scale 更精确，但是每个矩阵用一个 scale 精度也够，所以我们做了这个妥协。

对于 token 来说，问题在于不同 token 的激活值差别可能很大，如果强行让一个 batch 的 token 共用一个 scale，那么这个 scale 会极大程度上受到离群值的影响，导致精度变差。

事实上 ai 中的量化技术会把离群值找出来单独处理。

3

INT8 的范围是 $[-127, 127]$ （通常而言会人为采用对称范围），所以乘积最大绝对值就是 $127 \times 127 = 16129$ 。

INT16 的最大值是 32767，所以最坏情况下累加 3 项就会溢出。

使用最大规约长度 `MAX_D_MODEL=1024`，那么 $1024 \times 16129 = 16516096$ 。

INT32 上限是 2147483647，减一下发现余量为 2130967551，绰绰有余。